

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1

Total Marks:100

Annexure A

Assignment

Code of Conduct:

I Athisaya U (192571001) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Course Code & CSA11 – Object Oriented Analysis and Design Name

Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Campus Management System (AISCAMS)
Course Outcome(s) CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an AI-Enabled Smart Campus Management System (AISCAMS) that integrates academic, administrative, infrastructure, and student-support services through a centralized intelligent platform. The system shall enable students, faculty, administrators, security personnel, and maintenance staff to access campus services through role-based interfaces.

The system shall allow students to manage academic schedules, classroom bookings, campus facilities, events, digital services, and service requests. Faculty members shall be able to manage classes, attendance, academic activities, and classroom requirements. The system shall use intelligent recommendations to suggest suitable classrooms, facilities, and campus resources based on availability and usage patterns.

The system shall monitor campus infrastructure such as classrooms, laboratories, smart devices, parking areas, and maintenance equipment. IoT-enabled devices may provide information regarding room occupancy, environmental conditions, and equipment status. Maintenance personnel shall receive service requests and update repair status.

Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop appropriate UML models including Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall evaluate object

responsibilities and apply appropriate design principles and patterns to achieve modularity, maintainability, reusability, and scalability.

Constraints

The system shall clearly define the system boundary, actors, objects, classes, attributes, operations, and relationships. UML diagrams shall be consistent with one another, and the operations identified in the Class Diagram shall support the interactions represented in the Sequence and Activity Diagrams.

The design shall demonstrate appropriate use of encapsulation, abstraction, inheritance, polymorphism, cohesion, coupling, and responsibility assignment. Suitable design patterns shall be selected and justified based on the identified design problems.

The final model shall document assumptions and constraints and ensure that all UML models are complete, consistent, unambiguous, traceable, and feasible.

Tools Can Be Used

- StarUML
 - Visual Paradigm
 - Draw.io
 - PlantUML
 - Papyrus Tools
 - ArgoUML
 - Enterprise Architect
 - Lucidchart
-

Deliverables

To receive full credit, each submission must include the following:

1. **Problem Analysis and Requirement Summary**
2. **Actor Identification**
3. **Use Case Descriptions**
4. **Use Case Diagram**
5. **Object and Class Identification**
6. **Class Diagram**
7. **Sequence/Interaction Diagram(s)**
8. **Activity Diagram**
9. **State Transition Diagram**
10. **Package Diagram**
11. **Component Diagram**
12. **Deployment Diagram**
13. **Object Responsibility Analysis**
14. **Design Axioms/Principles and Design Pattern Justification**
15. **Assumptions and Constraints**
16. **Requirement-to-Use Case/Class Traceability**
17. **Implementation/Prototype and Results**
18. **Reflection on Design Decisions and Learning Outcomes**
19. **GitHub Upload**

QUESTION Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Campus Management System (AISCams).

ANSWER

AI-Enabled Smart Campus Management System

Object-Oriented Analysis, UML Design and Verified Prototype

Student	Registration Number	Course	Verification
Athisaya U	192571001	CSA11 - OOAD	170/170 tests passed

DESIGN POSITION AISCams uses explainable weighted ranking rather than an opaque machine-learning model. Every recommendation is traceable to hard eligibility filters and six displayed scoring factors.

1. Problem Analysis and Requirement Summary

AISCaMS addresses fragmentation across academic scheduling, resource booking, infrastructure monitoring, service support, campus events and administrative oversight. A unified object-oriented model is appropriate because the domain contains persistent entities with identity, behaviour and lifecycle rules: users act through roles, campus resources expose specialised capabilities, bookings reserve resources, requests move through controlled states, and IoT readings trigger observable events.

1.1 Functional requirements

- Authenticate users and present role-specific dashboards for students, faculty, administrators, security officers and maintenance staff.
- Search classrooms, laboratories, parking areas and smart devices using capacity, equipment, availability and location criteria.
- Generate explainable resource recommendations and retain explicit reasons for rejected candidates.
- Create bookings only after revalidating availability, capacity, equipment and time-conflict rules.
- Manage schedules, enrolments, attendance, campus events, registrations, digital services and notifications.
- Create service requests from user descriptions through automatic category, priority and SLA classification.
- Assign requests to maintenance personnel and enforce legal lifecycle transitions.
- Ingest IoT readings, classify severity, suppress duplicate alerts and automatically raise critical service tickets.
- Allow security officers to monitor parking and acknowledge active alerts.
- Provide administrators with utilisation, booking, service-level and infrastructure analytics.

1.2 Non-functional requirements

- Explainability: ranking factors, points and rejection reasons must be visible.
- Security: permissions are enforced in the service layer as well as the interface.
- Consistency: bookings and request transitions are revalidated before persistence.
- Maintainability: routes, services, repositories, domain objects and patterns are separated.
- Reliability: SQLite foreign keys, idempotent seed data and automated tests protect integrity.

- Usability: responsive dashboards and task-oriented screens expose the most relevant actions per role.
- Feasibility: the prototype runs locally with Python, Flask and SQLite without external services.

2. Actor Identification

Actor	Primary responsibilities
Student	Views schedules and attendance; searches and books resources; registers for events; submits and tracks service requests.
Faculty Member	Views teaching schedules; records attendance; books facilities; raises and tracks infrastructure requests.
Administrator	Manages users, resources and events; approves bookings; assigns tickets; reviews campus analytics.
Security Officer	Monitors IoT alerts and parking occupancy; acknowledges incidents requiring operational response.
Maintenance Staff	Views assigned work, updates request status and records equipment condition.
IoT Gateway	Publishes device readings that may update infrastructure state, raise alerts and create automatic requests.

Actor boundaries are enforced through permissions. The IoT gateway is modelled as a non-human external actor because it initiates the reading-ingestion use case without an interactive login session.

3. Use Case Descriptions

ID	Use case	Actor	Precondition	Outcome
UC-01	Authenticate user	Registered user	Valid credentials	Role dashboard displayed
UC-02	Recommend resource	Student/Faculty	Criteria entered	Ranked eligible resources and rejection reasons
UC-03	Book resource	Student/Faculty	Selected eligible slot	CONFIRMED or PENDING booking reference
UC-04	Submit service request	Student/Faculty	Resource and description supplied	Typed request with priority and SLA
UC-05	Assign request	Administrator	Request is NEW	Technician assigned; state becomes ASSIGNED
UC-06	Advance request	Maintenance Staff	Request assigned	Legal state transition recorded in history
UC-07	Publish IoT reading	IoT Gateway	Known operational device	Reading stored and classified
UC-08	Raise critical alert	System/IoT Gateway	Critical reading; no open duplicate	Alert, ticket and notifications created
UC-09	Acknowledge alert	Security Officer	Open alert	Alert marked ACKNOWLEDGED
UC-10	View analytics	Administrator	Authorised session	Utilisation, SLA and resource summaries

3.1 Principal alternate and exception flows

- Recommendation: an unavailable, undersized, unequipped or conflicting resource is excluded and returned with its rejection reason.
- Booking: the rules are checked again immediately before saving; a newly created conflict stops the transaction.

- Service request: unknown categories, invalid priorities or illegal state transitions return validation errors instead of corrupting history.
- IoT ingestion: a repeated critical reading reuses the open alert and suppresses a second service ticket.

3.2 Fully Dressed Use Case Specifications

The four specifications below cover the system's highest-risk workflows. They define observable success, exceptions and business rules so that the use-case, interaction, class and test models can be checked against the same contract.

3.2.1 / UC-02 - Recommend a Suitable Campus Resource

Specification field	Definition
Goal	Recommend a Suitable Campus Resource
Primary actor	Student or Faculty Member
Supporting actors	CampusFacade, RecommendationService, resource/booking/IoT repositories and a RecommendationStrategy
Trigger	The actor enters attendee count, time window, resource type, equipment and optional building preference.
Preconditions	The actor is authenticated and has VIEW_RECOMMENDATIONS permission; the time window and attendee count are valid.
Success postconditions	Eligible resources are returned in descending score order with factor points and reasons; rejected resources retain a specific rejection reason.

Main success scenario

Step	Actor and system interaction
1	The actor opens Smart Recommendations and supplies the search criteria.
2	CampusFacade validates permission and creates a RecommendationCriteria object.
3	RecommendationService loads bookable campus resources and bookings that overlap the requested slot.
4	The service applies the availability, capacity, equipment and booking-conflict hard filters.
5	The active strategy scores every eligible candidate using capacity fit, equipment match, live occupancy, utilisation, distance and building preference.
6	The service sorts candidates by descending score and limits the response to the requested result count.
7	The interface displays ranked recommendations, factor points, explanation text and the rejected-resource list.

Alternate and exception flows

Reference	System response
A1 - Status	A resource that is not AVAILABLE is rejected with its current status.
A2 - Capacity	A room below the attendee requirement is excluded; laboratory capacity is also limited by workstations.
A3 - Equipment	A missing, FAULTY or OUT_OF_SERVICE item produces a named equipment rejection.
A4 - Conflict	An overlapping confirmed or pending booking blocks the candidate for the requested slot.
A5 - Empty result	The system returns an empty ranking plus all recorded rejection reasons; it never invents a result.
E1 - Invalid access/input	Permission, date, attendee or strategy errors return a controlled 4xx response without changing stored data.

Business rules

- Hard eligibility filters always execute before scoring; a high soft score cannot override a safety or feasibility rule.

- The six normalised factor weights total 1.00, and the displayed score equals the sum of the displayed factor points.
- RecommendationStrategy is replaceable at runtime without changing the use-case service or interface contract.

3.2.2 / UC-03 - Book a Campus Resource

Specification field	Definition
Goal	Book a Campus Resource
Primary actor	Student or Faculty Member
Supporting actors	CampusFacade, BookingService, resource/user/booking repositories and the Observer subject
Trigger	The actor selects Book on an eligible recommendation and supplies a purpose.
Preconditions	The actor has BOOK_RESOURCE permission; the resource exists and the requested time window is in the future.
Success postconditions	A persisted booking reference is returned as CONFIRMED or, for a student laboratory booking, PENDING; relevant observers receive the booking event.

Main success scenario

Step	Actor and system interaction
1	The actor submits the selected resource, time window, attendees, purpose and required equipment.
2	BookingService reloads the user and resource rather than trusting the recommendation result.
3	The service revalidates resource status, bookability, capacity and installed working equipment.
4	BookingRepository checks the current database for an overlapping calendar entry.
5	The service creates the Booking domain object and applies the approval rule.
6	BookingRepository saves the booking and generates its BK reference.
7	The event bus notifies the owner and, when approval is required, the administrator; the interface shows the final state.

Alternate and exception flows

Reference	System response
A1 - Approval	A student's laboratory request is stored as PENDING for an administrator's decision.
A2 - Cancellation	The booking owner may cancel an active booking, releasing the slot; another user cannot cancel it.
E1 - Race/conflict	A conflict created after recommendation but before submission is detected by the second repository check and the booking is refused.
E2 - Rule failure	Unavailable, non-bookable, undersized or unequipped resources raise specific domain errors.
E3 - Invalid window	Past starts, end-before-start, or durations over eight hours are rejected before persistence.

Business rules

- Confirmed and pending bookings block overlapping reservations; rejected, cancelled and completed records do not block future slots.
- Adjacent bookings are permitted because overlap uses $\text{start} < \text{existing_end}$ and $\text{end} > \text{existing_start}$.
- Approval, cancellation and ownership rules are enforced in the application layer and verified by automated tests.

3.2.3 / UC-04 - Submit and Classify a Service Request

Specification field	Definition
Goal	Submit and Classify a Service Request
Primary actor	Student, Faculty Member or Security Officer
Supporting actors	CampusFacade, MaintenanceService, ServiceRequestFactory, request/resource repositories and Observer subject
Trigger	The actor reports a campus issue against a known resource.
Preconditions	The actor has SUBMIT_REQUEST permission; the resource exists; title and description satisfy validation rules.
Success postconditions	The correct concrete ServiceRequest subtype is saved in NEW state with category, priority, SLA deadline, source and an initial history entry.

Main success scenario

Step	Actor and system interaction
1	The actor selects a resource and submits a title and free-text description.
2	CampusFacade authorises the actor and delegates to MaintenanceService.
3	ServiceRequestFactory scans category keywords and selects Maintenance, Safety, IT Support or Housekeeping.
4	The factory evaluates escalation terms to derive LOW, MEDIUM, HIGH or CRITICAL priority.
5	The selected subclass applies its own SLA matrix and calculates sla_due_at.
6	ServiceRequestRepository stores the request and NEW history record under an SR reference.
7	The event bus creates the appropriate notification/audit reaction and the interface displays the classification and SLA.

Alternate and exception flows

Reference	System response
A1 - Explicit values	A valid explicit category or priority overrides keyword classification while still using the subtype SLA matrix.
A2 - Escalation	Urgent safety language raises priority, which shortens the SLA according to the concrete request class.
A3 - Default	If no specialised keyword matches, the factory creates a MaintenanceRequest.
E1 - Invalid request	Missing resource, short text, unknown category or invalid priority returns a controlled validation response.
E2 - Assignment	A NEW request remains unassigned until an administrator selects a MaintenanceStaff user.

Business rules

- Object creation is centralised in ServiceRequestFactory so routes and services never construct concrete request subclasses directly.
- Every subtype owns the CATEGORY, SLA_MATRIX and handling_team behaviour used polymorphically after creation.
- All later status changes must pass ServiceRequest.can_transition_to() and are appended to request history.

3.2.4 / UC-08 - Process a Critical IoT Reading and Raise an Alert

Specification field	Definition
Goal	Process a Critical IoT Reading and Raise an Alert
Primary actor	IoT Gateway
Supporting actors	IoTService, IoTReading, repositories, MaintenanceService, ServiceRequestFactory, Subject and alert/notification observers
Trigger	A registered smart device publishes a metric value through POST /api/iot/readings.
Preconditions	The device exists, is a SmartDevice and supplies a supported metric with a numeric value.
Success postconditions	The reading is persisted; a new critical incident creates one open alert, one CRITICAL request assigned to the least-loaded technician, and notifications for operational roles.

Main success scenario

Step	Actor and system interaction
1	The gateway submits device identity, metric, value and optional recorded time.
2	IoTService verifies the device type and creates an IoTReading.
3	IoTReading.classify() derives NORMAL, WARNING or CRITICAL from the metric-specific threshold.
4	IoTRepository persists the reading before any secondary action.
5	For a critical value, IoTService searches for an existing open alert for the monitored resource and metric.
6	When no duplicate exists, MaintenanceService asks the Factory to create a CRITICAL request and selects the least-loaded technician.
7	Subject notifies AlertObserver, NotificationObserver and AuditTrailObserver; the alert links to the created ticket.
8	The API returns HTTP 201 with the reading, alert, request and duplicate_suppressed result.

Alternate and exception flows

Reference	System response
A1 - Normal	The reading is stored and returned without an alert or service request.
A2 - Warning	The reading is stored for monitoring; no automatic ticket is raised.
A3 - Duplicate critical	The existing open alert is reused and duplicate_suppressed is true, preventing a second ticket.
A4 - Side effect	Parking occupancy and device-status readings also update the monitored resource's operational state.
E1 - Invalid source	Unknown devices, non-smart resources, unsupported metrics or non-numeric values are rejected without partial escalation.

Business rules

- Thresholds belong to the IoTReading domain model, while orchestration and duplicate suppression belong to IoTService.
- A critical reading is saved before observers run, preserving evidence even if a downstream reaction fails.
- Only one unresolved alert/request pair is maintained for the same resource and metric incident.

4. Use Case Diagram

The system boundary contains academic, booking, support, infrastructure and administrative use cases. Human actors remain outside the boundary, while the IoT gateway includes automatic alert and request creation when a reading is critical.

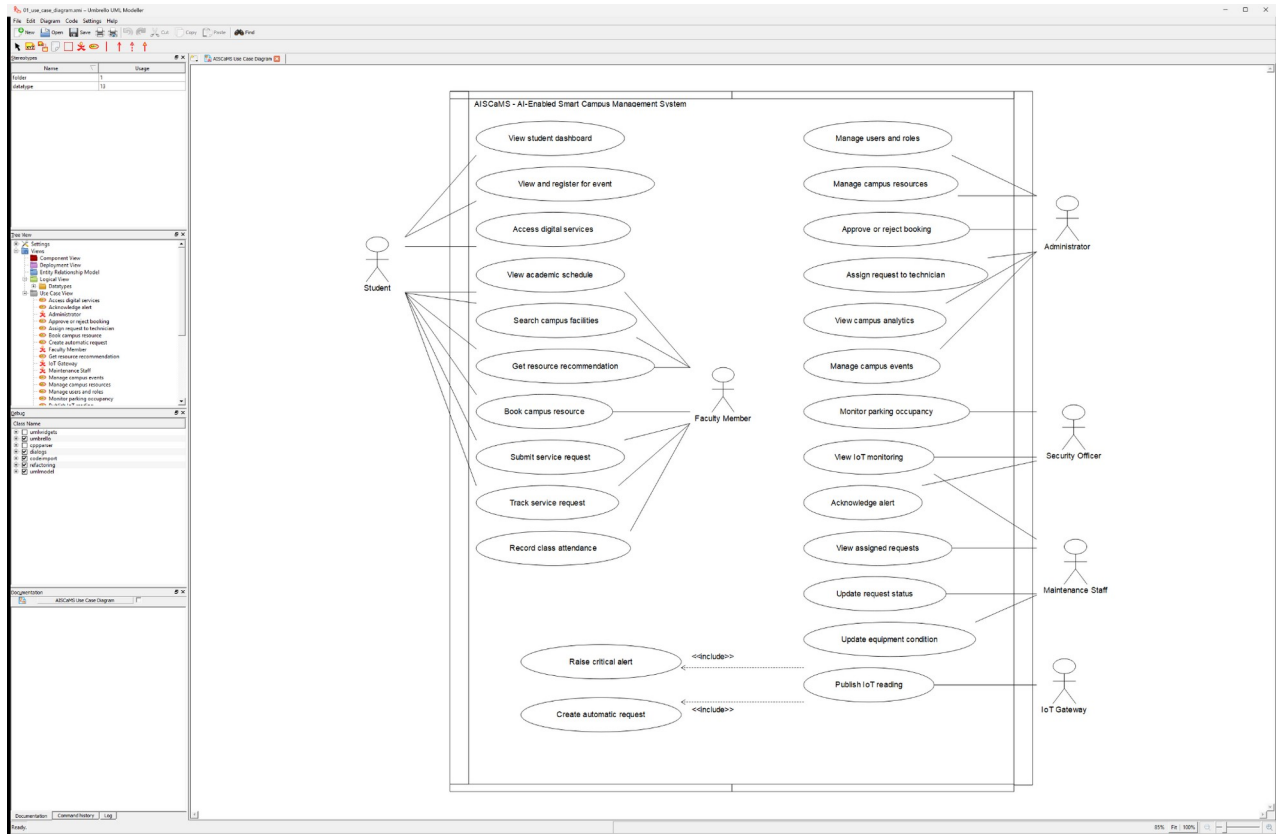


Figure 1. AISCAMS Use Case Diagram created in Umbrello

5. Object and Class Identification

Area	Classes	Purpose
Identity	User, Student, Faculty, Administrator, SecurityOfficer, MaintenanceStaff	Role-specific permissions and dashboards
Resources	Building, CampusResource, Classroom, Laboratory, ParkingArea, SmartDevice	Bookability, capacity, equipment and live state
Academic	Schedule, AttendanceRecord	Teaching, enrolment and attendance
Transactions	Booking, CampusEvent, Notification	Reservations, event participation and user messages
Support	ServiceRequest and four concrete request subclasses	Priority, SLA, assignment and lifecycle
IoT	IoTReading	Metric classification and intervention decision
Patterns	RecommendationStrategy, ServiceRequestFactory, Subject/Observer	Replaceable policy, controlled creation and event response
Application	CampusFacade, services and repositories	Workflow orchestration and persistence separation

5.1 Identity, Academic and Transaction Class Catalogue

The catalogue records the state and behaviour that make each class an object-oriented abstraction rather than a passive database row. Only implementation-significant attributes and operations are listed.

Class / role	Responsibility and important state	Core operations / behaviour
User (abstract)	Common identity and account state: id, username, full name, email, department, phone and is_active.	role(), job_title(), permissions(), landing_page(), has_permission(), dashboard_sections(), activate(), deactivate(), to_dict().
Student	Represents an enrolled learner and specialises role permissions and the student dashboard.	role(), permissions(), landing_page().
Faculty	Represents teaching staff with booking, attendance and service-request privileges.	role(), permissions(), landing_page().
Administrator	Represents the role that manages users, resources, approvals, tickets, events and analytics.	role(), permissions(), landing_page().
SecurityOfficer	Represents operational staff who monitor parking/IoT incidents and acknowledge alerts.	role(), permissions(), landing_page().
MaintenanceStaff	Represents a technician who owns assigned requests and updates valid request states.	role(), permissions(), landing_page().
Building	Identifies a campus building by code/name and stores its walking-distance reference value.	to_dict(); aggregates campus resources.
Schedule	Stores course code, day, start/end time, faculty ownership and enrolment context.	is_today(); supplies the parent context for attendance.
AttendanceRecord	Records one student's PRESENT/ABSENT/LATE status for a scheduled session date.	is_present(); validates attendance state.
Booking	Protects reference, resource/user, purpose, slot, attendees, approval and lifecycle state.	overlaps(), blocks_calendar(), confirm(), reject(), cancel(), complete(), duration_hours().
CampusEvent	Stores an event's title, time, capacity, registration count and publication state.	is_upcoming(), seats_left(); protects capacity.
Notification	Represents a categorised user message and its read/unread state.	mark_read(); supports Observer reactions.

5.2 Resource, Support and IoT Class Catalogue

Class / role	Responsibility and important state	Core operations / behaviour
CampusResource (abstract)	Common resource identity, building/floor, capacity, status, utilisation, equipment and walking distance.	resource_type(), is_available(), is_bookable(), has_equipment(), missing_equipment(), matches_capacity(), set_status(), set_equipment_condition(), summary().
Classroom	Adds seating type and board type to an ordinary bookable teaching space.	is_bookable(), summary().
Laboratory	Adds laboratory type, safety level and workstation count to capacity rules.	matches_capacity(); checks both seats and workstations; summary().
ParkingArea	Tracks zone, total slots and occupied slots for live parking visibility.	free_slots(), occupancy_rate(), update_occupancy().
SmartDevice	Represents device type, firmware and online/operational condition.	is_operational(), summary(); publishes IoT readings.
IoTReading	Stores metric, numeric value, severity and recorded time for one smart device.	classify(), is_critical(), requires_intervention(); applies metric thresholds.
ServiceRequest (abstract)	Protects ticket, resource, description, category, priority, status, requester, assignee, source and SLA.	can_transition_to(), transition_to(), assign(), escalate(), sla_state(), is_open(), handling_team().
MaintenanceRequest	Concrete general-facilities request with its own priority-to-SLA matrix.	handling_team(); inherits lifecycle behaviour.
SafetyRequest	Concrete health/security request with shorter safety response targets.	handling_team(); inherits lifecycle behaviour.
ITSupportRequest	Concrete technology request for networks, projectors and digital equipment.	handling_team(); inherits lifecycle behaviour.
HousekeepingRequest	Concrete cleaning and hygiene request with category-specific response targets.	handling_team(); inherits lifecycle behaviour.

5.3 Pattern and Application Class Catalogue

Class / component	Responsibility and important state	Core operations / behaviour
RecommendationStrategy	Abstract policy contract for ranking eligible RecommendationCandidate objects.	score(candidate, criteria), rank(candidates, criteria).
WeightedRankingStrategy	Uses six declared weights and retains per-factor points and explanation text.	score(), rank(); capacity, equipment, occupancy, utilisation, distance and preferred-building factors.
Strategy variants	ProximityFirstStrategy and UtilisationBalancingStrategy change ranking priorities without changing clients.	Override factor weights/scoring behind the same strategy contract.
ServiceRequestFactory	Centralises category/priority classification and concrete request subtype creation.	create(), create_from_reading(), classify_category(), classify_priority().
Subject / Observer	Subject owns registered event observers; concrete observers react to workflow events independently.	attach(), detach(), notify(); NotificationObserver, AlertObserver and AuditTrailObserver.update().
RepositoryRegistry	Provides one coherent persistence boundary and shared SQLite connection to services.	Exposes ten concrete repositories and transaction-safe access.
Concrete repositories	Translate users, resources, bookings, schedules, requests, IoT, alerts, notifications and events between rows and domain objects.	get(), list(), add(), update() and specialised conflict/history/analytics queries.
CampusFacade	Coarse-grained authorised entry point used by Flask routes and role dashboards.	recommend_resources(), book_resource(), submit_request(), assign_request(), record_iot_reading(), dashboard_for().
Application services	Recommendation, Booking, Maintenance, IoT and Analytics services orchestrate one workflow family each.	Validate permissions and invariants, coordinate repositories/patterns and emit domain events.

5.4 Relationship decisions

- Inheritance separates stable abstractions from role-, resource- and request-specific behaviour.
- Associations connect users to bookings, requests, schedules, events and notifications with explicit multiplicities.
- Aggregation models a building containing independently identifiable campus resources.
- Composition is used where child records do not have a meaningful lifecycle without their owner, such as schedule attendance and smart-device readings.
- Dependencies identify creation and ranking policies without forcing services to construct concrete domain types directly.

[illegible]

KEY MULTIPLICITIES One user can create many bookings and requests; one resource can receive many bookings and requests; one smart device can publish many readings; one schedule owns many attendance records.

7. Sequence / Interaction Diagrams

The three interaction models show the most important end-to-end workflows. They follow the implemented layering: actor to CampusFacade, facade to service, service to repository/pattern, and a result returned to the actor.

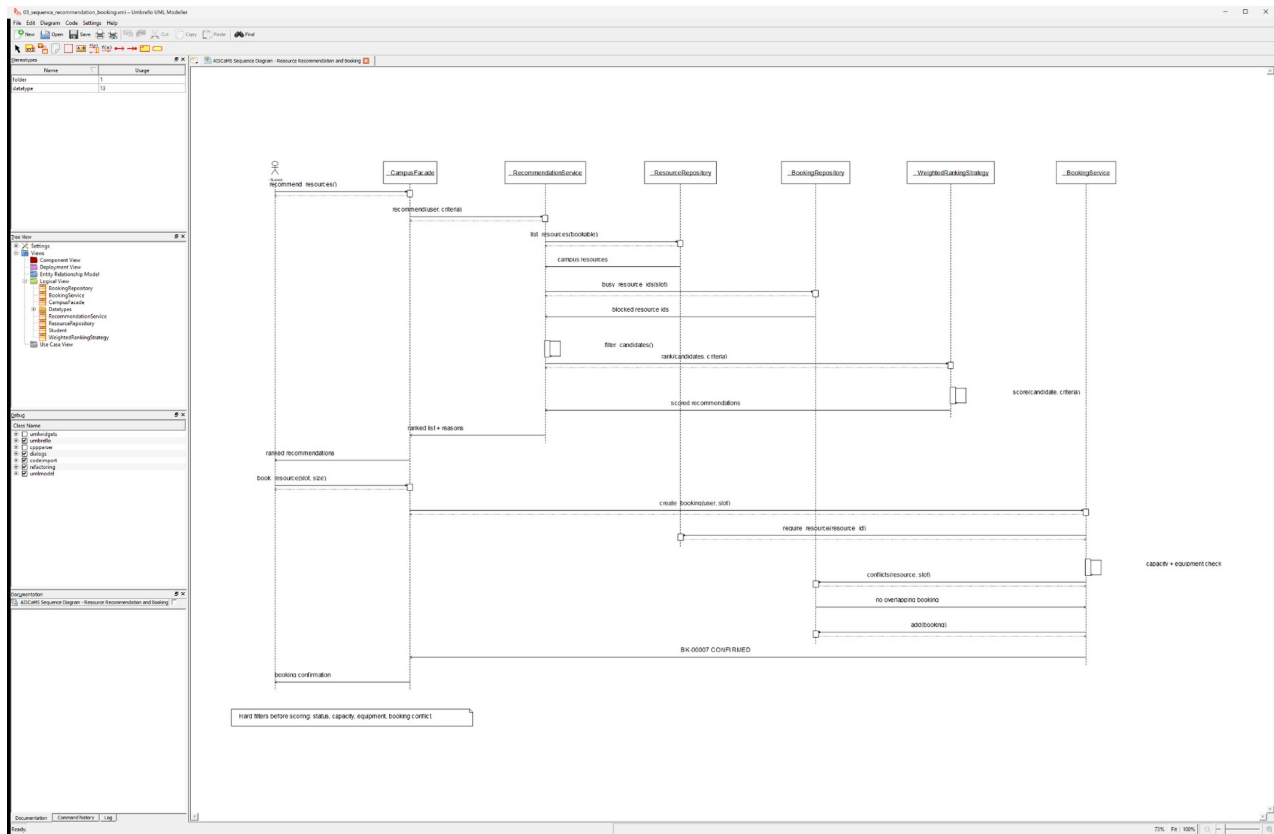


Figure 3. Resource Recommendation and Booking Sequence

The recommendation service obtains bookable resources, excludes blocked candidates, delegates scoring to WeightedRankingStrategy and returns explanations. BookingService then revalidates the selected resource before persisting booking BK-00007.

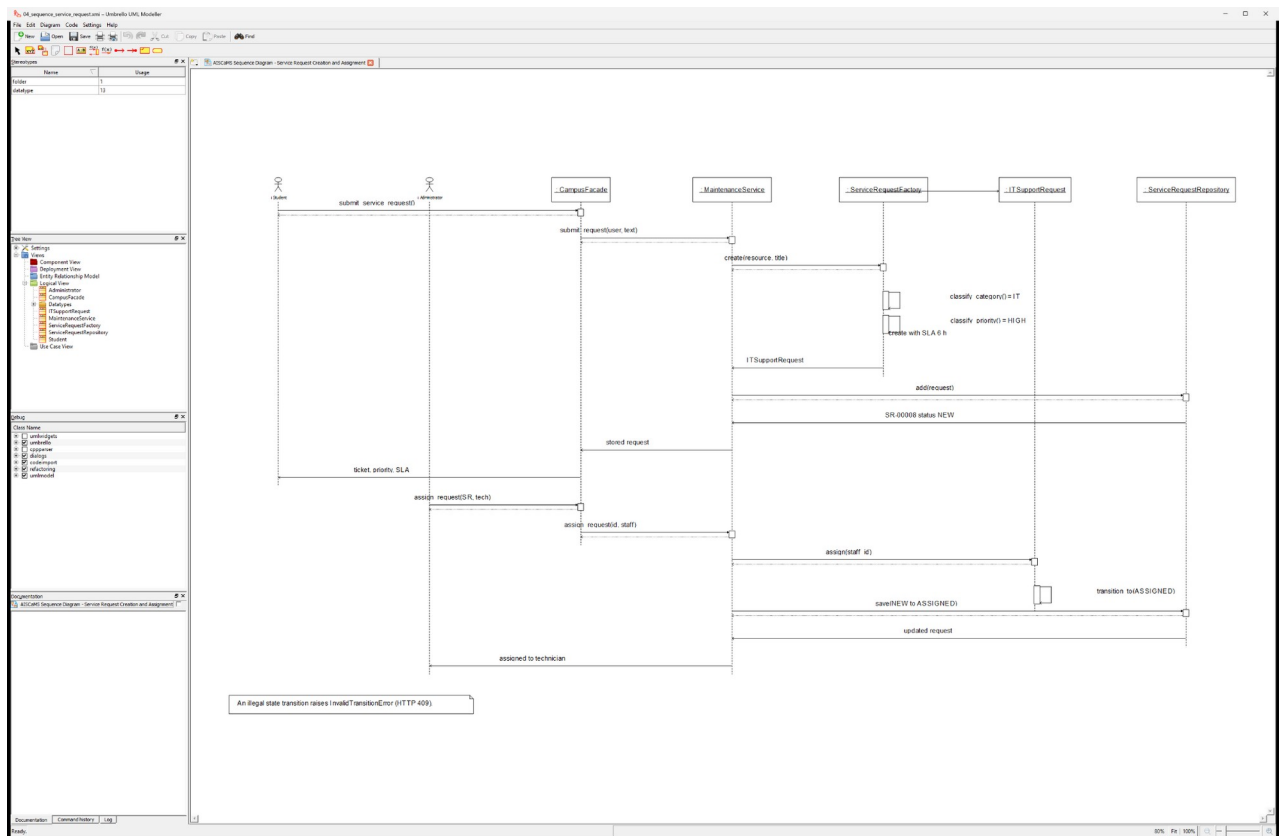


Figure 4. Service Request Creation and Assignment Sequence

ServiceRequestFactory selects ITSupportRequest, derives HIGH priority and applies the six-hour SLA. The administrator subsequently assigns the request, producing the legal NEW-to-ASSIGNED transition.

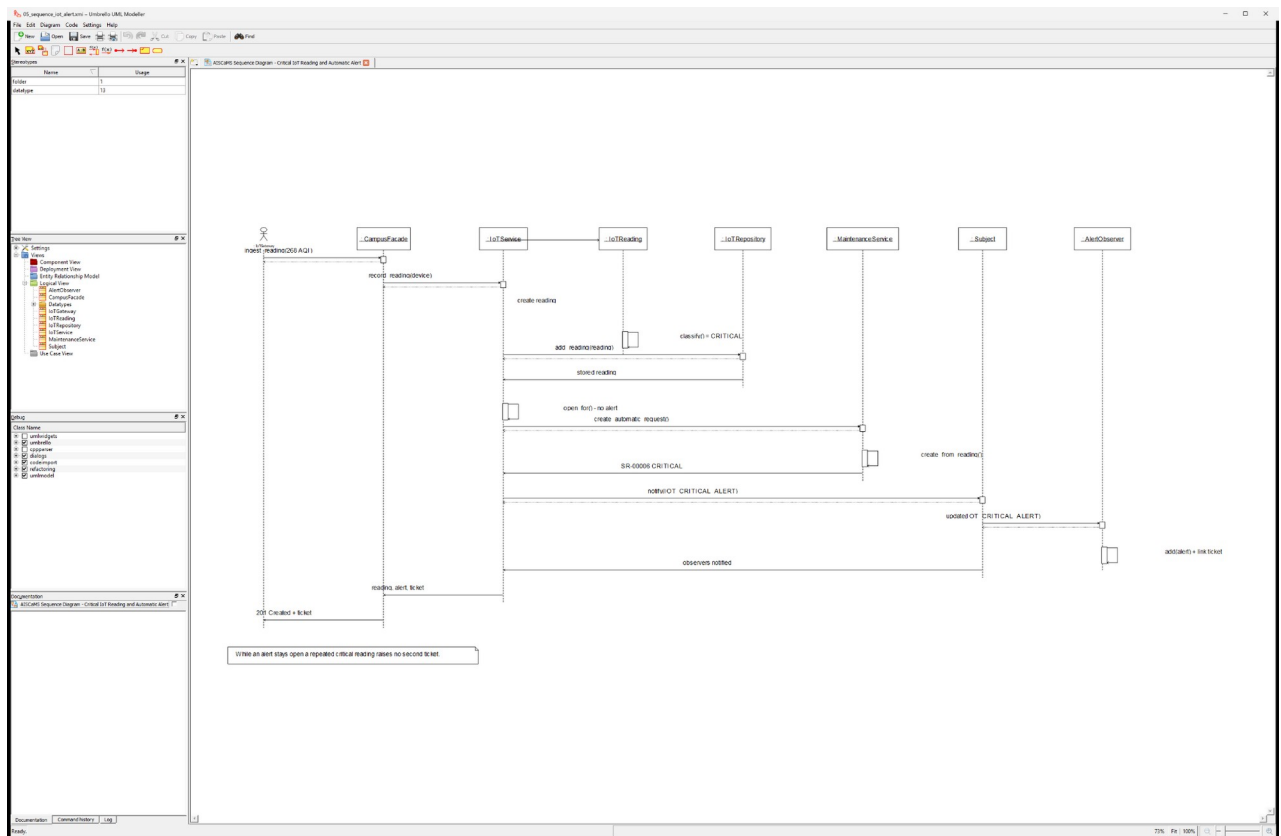


Figure 5. Critical IoT Reading and Automatic Alert Sequence

A critical AQI reading is persisted first. When no matching alert is open, MaintenanceService creates a CRITICAL request and the Observer bus raises notifications and an alert linked to the ticket.

8. Activity Diagram

The activity model captures the four hard filters, scoring path and booking approval decision. Rejected candidates retain reasons and remain visible as filtered-out resources, while eligible candidates are ranked and explained.

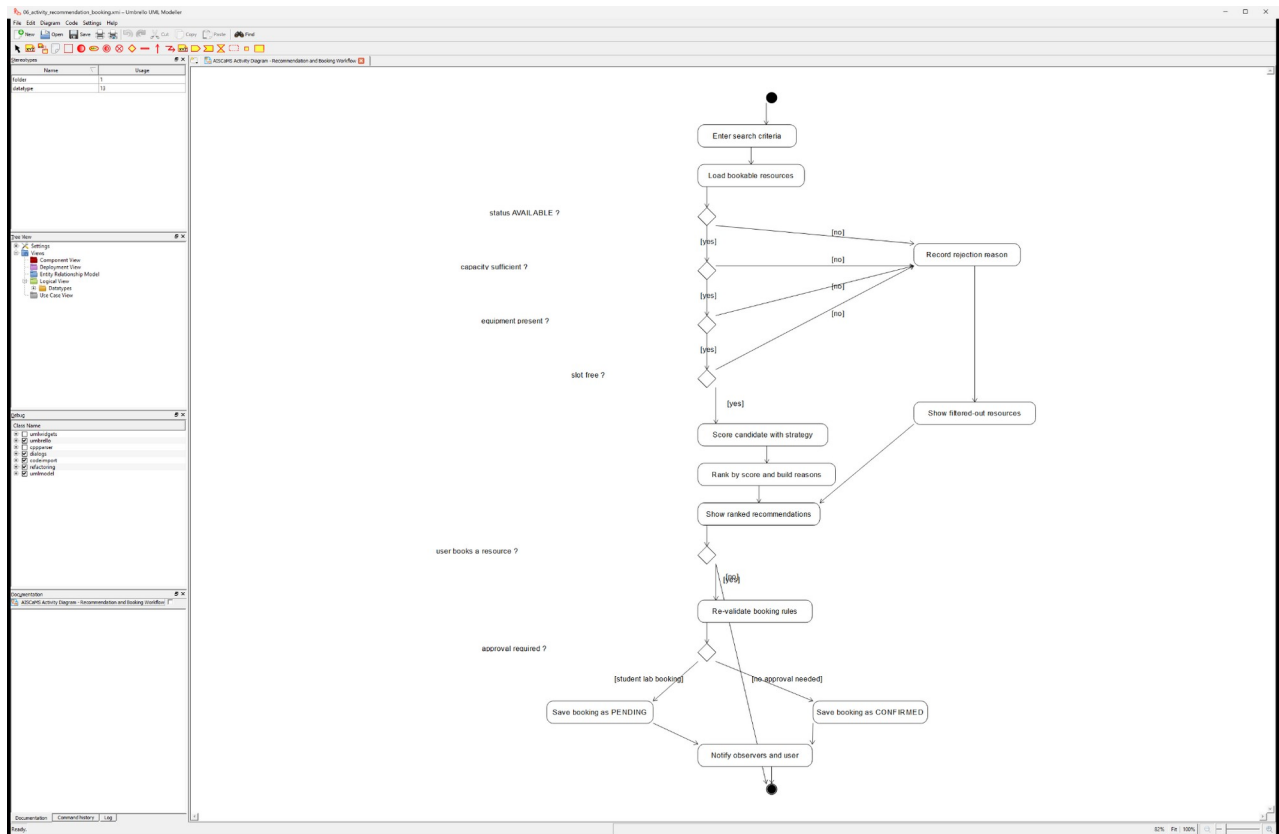


Figure 6. Recommendation and Booking Activity Diagram

9. State Transition Diagram

Service requests use an explicit state machine. CLOSED and REJECTED are terminal states. A RESOLVED request may be reopened to IN_PROGRESS when verification fails, preserving a controlled audit trail.

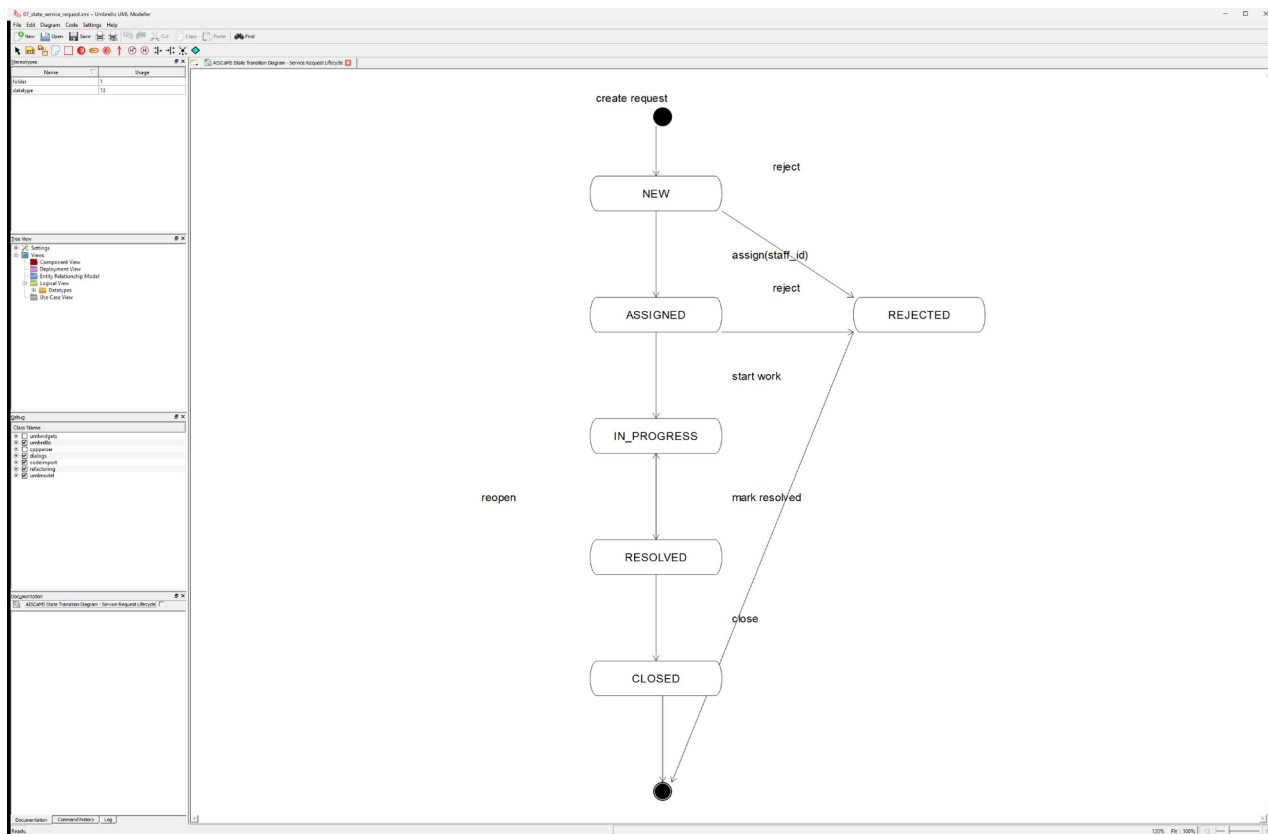


Figure 7. Service Request Lifecycle State Diagram

10. Package Diagram

The package model applies dependency direction from interface and security modules toward services, then patterns/repositories/domain, with database and utility packages at the infrastructure edge.

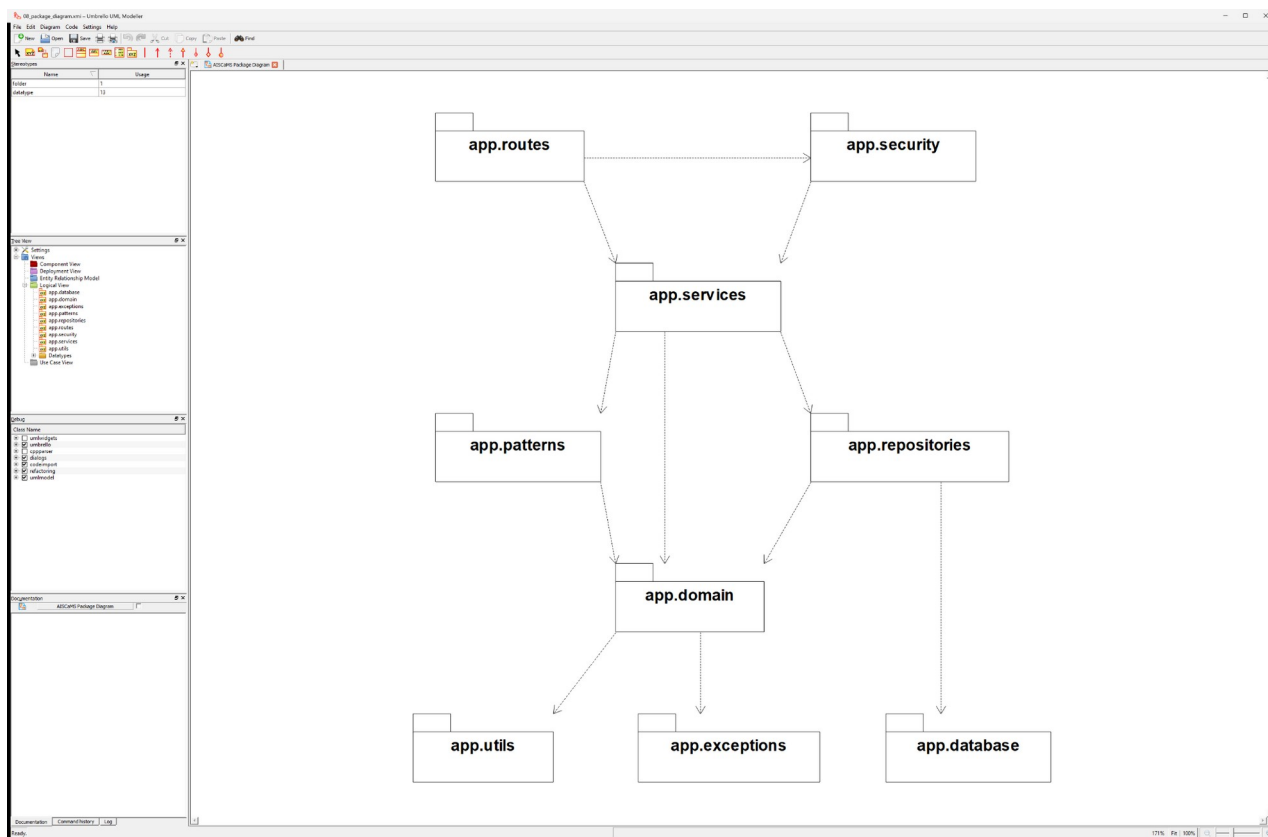


Figure 8. AISCaMS Package Dependency Diagram

11. Component Diagram

The component view separates the web UI and IoT gateway from Flask blueprints, the facade, application services, runtime design-pattern components, the domain model and repository-backed SQLite storage.

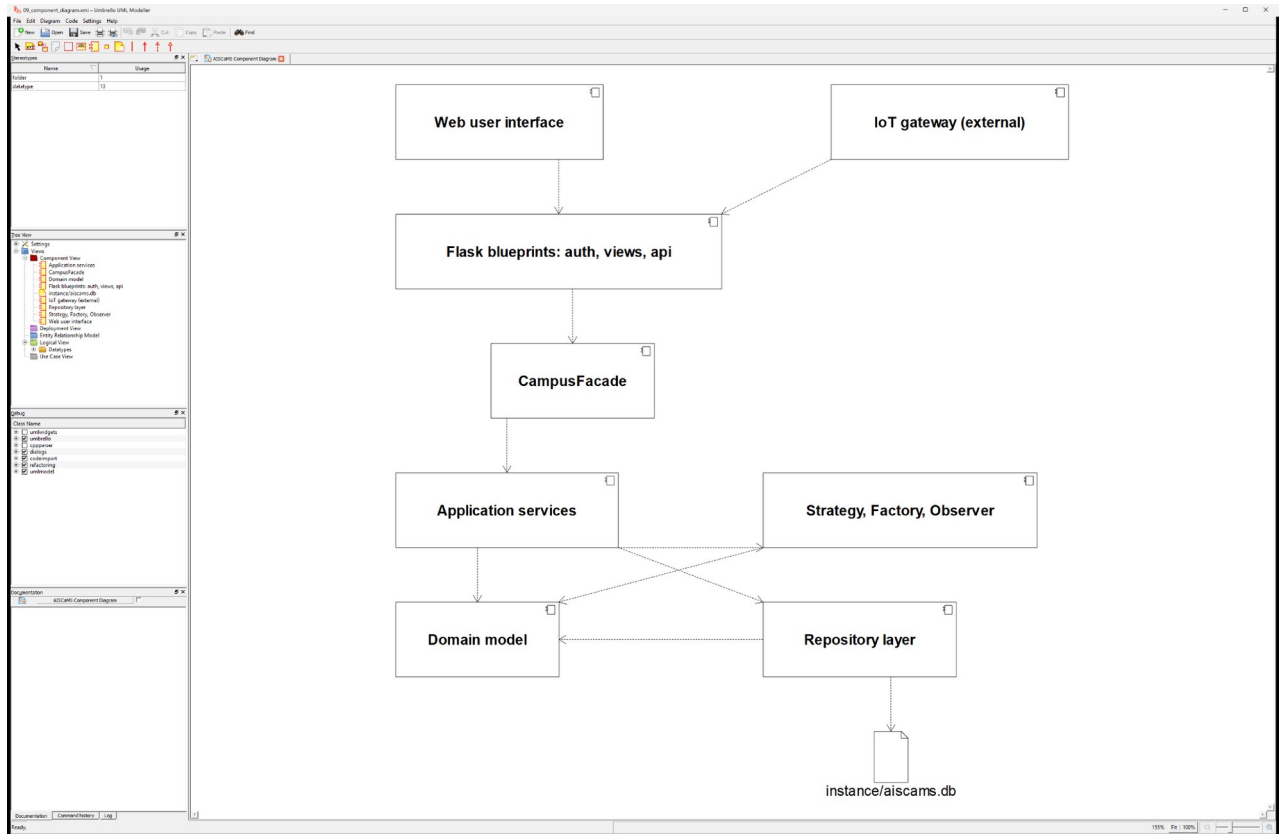


Figure 9. AISCams Component Diagram

12. Deployment Diagram

The prototype is deployed on a Windows or Linux campus host. Browsers access Flask over HTTP, IoT gateways submit readings to the API, and the Python process persists data through file I/O to the local SQLite database.

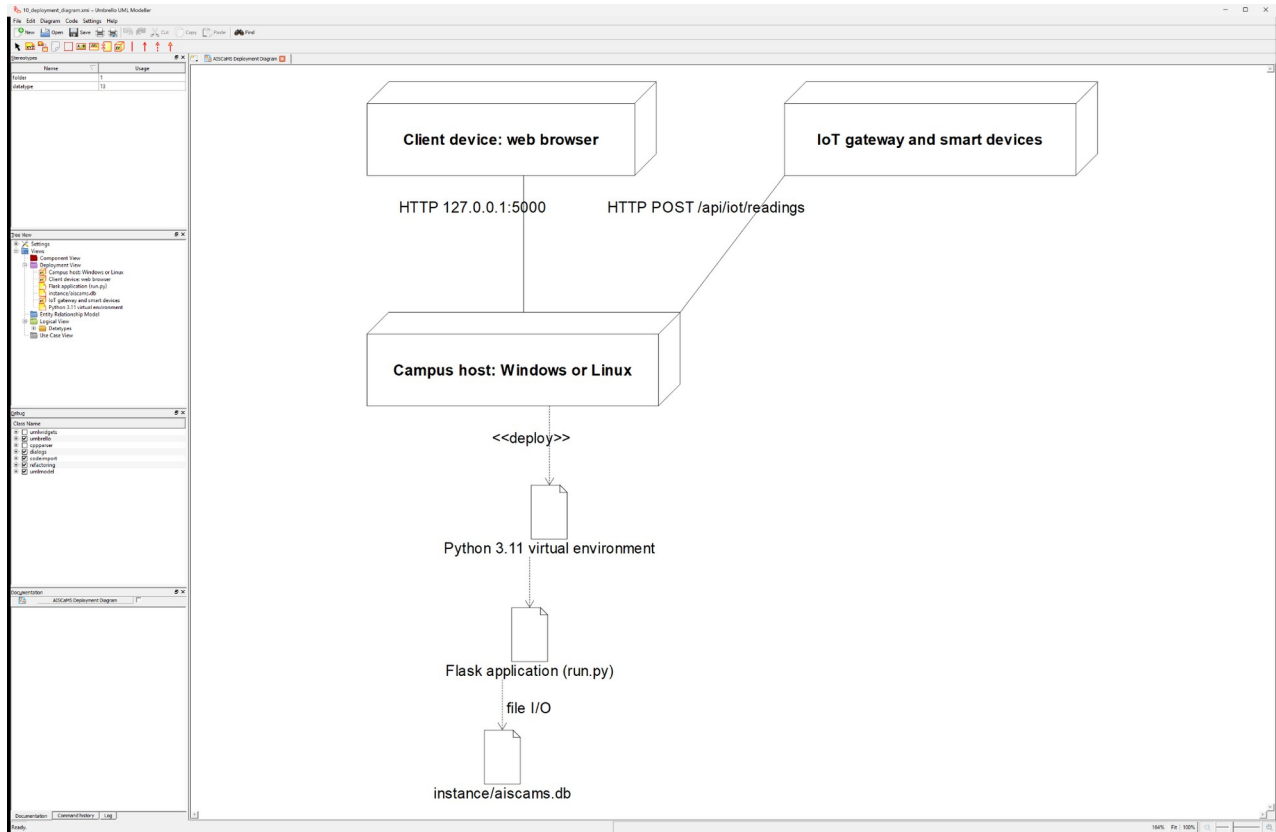


Figure 10. AISCams Deployment Diagram

13. Object Responsibility Analysis

Class / component	Responsibilities	Collaborators
CampusFacade	Expose coarse-grained campus workflows; construct dashboard payloads	Application services; RepositoryRegistry
RecommendationService	Apply hard filters; create candidates; request ranking; preserve rejection reasons	ResourceRepository; BookingRepository; IoTRepository; Strategy
WeightedRankingStrategy	Score six normalised factors and produce explanations	Candidate; RecommendationCriteria
BookingService	Validate and persist bookings; emit booking events	BookingRepository; ResourceRepository; Subject
MaintenanceService	Submit, assign and advance requests; compute SLA overview	Factory; RequestRepository; UserRepository; Subject
ServiceRequestFactory	Classify category/priority and instantiate the correct request subtype	Concrete ServiceRequest subclasses; IoTReading
ServiceRequest	Protect state, assignment and SLA invariants	MaintenanceStaff; Request history
IoTService	Store readings; suppress duplicates; create critical tickets; notify observers	IoT/Alert repositories; MaintenanceService; Subject
Subject	Register observers and broadcast domain events	NotificationObserver; AlertObserver; AuditTrailObserver
RepositoryRegistry	Provide cohesive repository access over one connection	Ten concrete repositories; SQLite

Responsibilities remain cohesive: domain objects protect their own invariants, services coordinate use cases, repositories translate persistence, and the facade protects the web layer from subsystem complexity.

14. Design Axioms, Principles and Pattern Justification

Principle	Application in AISCaMS
Encapsulation	Private state and transition methods prevent invalid request, booking and resource mutations.
Abstraction	User, CampusResource, ServiceRequest, RecommendationStrategy and Observer express stable contracts.
Inheritance	Role, resource and request subclasses reuse shared state while specialising permissions and behaviour.
Polymorphism	Dashboard sections, request SLA rules and recommendation strategies vary behind common interfaces.
High cohesion	Each service owns one workflow family; each repository owns one persistence family.
Low coupling	Routes depend on CampusFacade; services depend on repository interfaces and pattern contracts.
Open/Closed Principle	New ranking strategies and observers can be introduced without modifying client workflows.
Dependency Inversion	Policy objects and repositories are injected into services instead of being created inside methods.

14.1 Runtime design patterns

Pattern	Implementation	Justification
Strategy	WeightedRankingStrategy plus proximity-first and utilisation-balancing variants	Swaps ranking policy while preserving recommendation workflow and explanations.
Factory	ServiceRequestFactory	Centralises concrete subtype, category, priority and SLA creation.
Observer	Subject with notification, alert and audit observers	Decouples workflow completion from downstream reactions.
Repository	Ten repositories behind RepositoryRegistry	Keeps SQL and row reconstruction outside domain/services.
Facade	CampusFacade	Provides one protected entry point for routes and dashboards.

Code excerpt A - Explainable ranking weights

```

109 class WeightedRankingStrategy(RecommendationStrategy):
110     """Weighted sum of normalised factors; every subclass tunes the weights."""
111
112     key = "balanced"
113     label = "Balanced smart match"
114     description = ("Weights capacity fit, equipment, live occupancy, historical "
115                  "utilisation, preferred building and walking distance.")
116     weights = {
117         "capacity_fit": 0.25,
118         "equipment_match": 0.20,
119         "live_occupancy": 0.15,
120         "past_utilisation": 0.15,
121         "walking_distance": 0.15,
122         "building_preference": 0.10,
123     }

```

Each factor has a declared weight; the displayed score is the exact sum of factor points rather than a separately generated explanation.

Code excerpt B - Factory classification rules

```

63 def classify_category(text):
64     """Pick a category from the reported text using keyword rules."""
65     haystack = " %s " % (text or "").lower()
66     for category, keywords in CATEGORY_KEYWORDS:
67         for keyword in keywords:
68             if keyword in haystack:
69                 return category
70     return "MAINTENANCE"
71
72 @staticmethod
73 def classify_priority(text, category):
74     """Derive a priority from the category plus escalation keywords."""
75     haystack = " %s " % (text or "").lower()
76     priority = BASE_PRIORITY.get(category, PRIORITY_MEDIUM)
77     escalations = [word for word in ESCALATION_KEYWORDS if word in haystack]
78     if escalations:
79         index = min(PRIORITY_ORDER[priority] + 1, PRIORITY_ORDER[PRIORITY_CRITICAL])
80         priority = PRIORITIES[index]
81     if category == "SAFETY" and escalations:
82         priority = PRIORITY_CRITICAL
83     return priority, escalations

```

Code excerpt C - Explicit legal request transitions

```

23 ALLOWED_TRANSITIONS = {
24     STATUS_NEW: (STATUS_ASSIGNED, STATUS_REJECTED),
25     STATUS_ASSIGNED: (STATUS_IN_PROGRESS, STATUS_REJECTED),
26     STATUS_IN_PROGRESS: (STATUS_RESOLVED,),
27     STATUS_RESOLVED: (STATUS_CLOSED, STATUS_IN_PROGRESS),
28     STATUS_CLOSED: (),
29     STATUS_REJECTED: (),

```

Code excerpt D - Critical IoT workflow and duplicate suppression

```

67         result = {"reading": stored, "alert": None, "request": None,
68                  "duplicate_suppressed": False}
69         if not stored.requires_intervention():
70             return result
71
72         # A still-open alert for the same resource/metric must not raise a
73         # second ticket for the same underlying problem.
74         existing = self._alerts.open_for(stored.resource_id or device.id, stored.metric)
75         if existing:
76             result["alert"] = existing
77             result["duplicate_suppressed"] = True
78             return result
79
80         request = self._maintenance.create_automatic_request(
81             stored, resource_label=target.code if target else device.code)
82         payload = {
83             "reading": stored,
84             "request_id": request.id,
85             "recipients": self._watchers() + ([request.assigned_to] if request.assigned_to
86             else []),
87             "title": "Critical IoT alert on %s" % (stored.resource_code or device.code),
88             "message": "%s Ticket %s was raised automatically (%s priority)."
89                       % (stored.describe(), request.ticket, request.priority),
90             "entity_type": "ALERT",
91             "entity_id": stored.id,
92         }
93         self._bus.notify(IOT_CRITICAL_ALERT, payload)
94         alert_id = payload.get("alert_id")
95         if alert_id:
96             self._alerts.link_request(alert_id, request.id)
97             result["alert"] = self._alerts.get(alert_id)
98         result["request"] = request
99         return result

```

Code excerpt E - Facade wiring of repositories, observers and services

```

22 class CampusFacade:
23     """Coordinates the AISCaMS subsystems behind a single interface."""
24
25     def __init__(self, connection):
26         self.repos = RepositoryRegistry(connection)
27
28         # Observer pattern: one bus, three independent observers.
29         self.bus = Subject()
30         self.notification_observer = self.bus.attach(
31             NotificationObserver(self.repos.notifications))
32         self.alert_observer = self.bus.attach(AlertObserver(self.repos.alerts))
33         self.audit_observer = self.bus.attach(AuditTrailObserver())
34
35         self.bookings = BookingService(self.repos.bookings, self.repos.resources,
36                                       self.repos.users, self.bus)
37         self.maintenance = MaintenanceService(self.repos.requests, self.repos.resources,
38                                              self.repos.users, self.bus)
39         self.recommendations = RecommendationService(self.repos.resources,
40                                                    self.repos.bookings, self.repos.iot)
41         self.iot = IoTService(self.repos.iot, self.repos.alerts, self.repos.resources,
42                             self.repos.users, self.maintenance, self.bus)
43         self.analytics = AnalyticsService(self.repos.resources, self.repos.bookings,
44                                         self.repos.requests, self.repos.users,
45                                         self.repos.iot, self.repos.alerts,
46                                         self.repos.events)

```

15. Assumptions and Constraints

15.1 Assumptions

ID	Assumption
A1	Users, devices, buildings and resources are registered before operational workflows begin.
A2	Room equipment inventories and capacities are maintained accurately by administrators.

ID	Assumption
A3	IoT values are trusted after device identity validation; device transport is simulated through HTTP in the prototype.
A4	A single campus time zone is used; the database stores local-time values consistently.
A5	A current open alert represents the same unresolved resource/metric incident and suppresses duplicates.
A6	The weighted strategy is an explainable decision policy and is not represented as trained machine learning.

15.2 Constraints

ID	Constraint
C1	Prototype stack is restricted to Flask, SQLite, server-rendered HTML, CSS and vanilla JavaScript.
C2	SQLite supports the single-host academic prototype; distributed production deployment would require a server database.
C3	No real campus identity provider, payment gateway or physical IoT broker is required for demonstration.
C4	Authorisation is role- and permission-based; passwords use Werkzeug hashing and demo credentials are non-production.
C5	All UML views must remain consistent with implemented classes, operations and workflow order.
C6	Recommendation factors remain deterministic so that results are testable and auditable.

16. Requirement-to-Use Case/Class Traceability

16.1 Forward Design Traceability

Requirement	Use case	Realising classes/services	View / API evidence
FR-01 Role access	UC-01	User hierarchy, permissions, CampusFacade	/login and /dashboard
FR-02 Facility search	UC-02	CampusResource hierarchy, ResourceRepository	/facilities; GET /api/resources
FR-03 Explainable recommendation	UC-02	RecommendationService, RecommendationStrategy	/recommendations; POST /api/recommendations
FR-04 Conflict-safe booking	UC-03	Booking, BookingService, BookingRepository	/bookings; POST /api/bookings
FR-05 Academic management	Dashboard	Schedule, AttendanceRecord, ScheduleRepository	/schedule; POST /api/attendance
FR-06 Service request creation	UC-04	Factory, request hierarchy, MaintenanceService	/requests; POST /api/requests
FR-07 Assignment/state	UC-05/06	MaintenanceService, ServiceRequest state rules	POST /api/requests/<id>/assign and /status
FR-08 IoT ingestion/ticket	UC-07/08	IoTReading, IoTService, observers	POST /api/iot/readings
FR-09 Alert acknowledgement	UC-09	SecurityOfficer, AlertRepository	POST /api/alerts/<id>/acknowledge
FR-10 Analytics	UC-10	AnalyticsService and repositories	/admin; GET /api/admin/analytics
NFR-01 Explainability	UC-02	ScoredRecommendation factors/reasons	Score bars, factor points and rejected list
NFR-02 Integrity/audit	UC-03-09	Schema constraints, repositories, AuditTrailObserver	History, notifications and constrained writes

16.2 Verification Traceability

Requirement	Automated evidence	Visual evidence	Result
FR-01	test_api_and_access.py: role-login, permission and forbidden-route tests	Figure 11; Figure 17	PASS
FR-02	test_resources.py and test_api_and_access.py: type, status, capacity and resource API tests	Figure 12	PASS
FR-03	test_recommendations.py: hard filters, factor totals, strategy variants and reasons	Figure 12	PASS
FR-04	test_bookings.py: overlap, capacity, equipment, approval, ownership and cancellation	Figure 13	PASS
FR-05	test_schedules_events_notifications.py and API attendance authorisation tests	Figure 11	PASS
FR-06	test_service_requests.py: factory categories, escalation, subtype SLA and validation	Figure 14	PASS
FR-07	test_service_requests.py: assignment ownership and every legal/illegal transition	Figure 16	PASS
FR-08	test_iot_and_alerts.py: thresholds, persistence, auto-ticket, assignment and duplicate suppression	Figure 15	PASS
FR-09	test_iot_and_alerts.py and API tests: role-controlled acknowledgement	Figure 15	PASS
FR-10	test_observer_and_dashboards.py: analytics payloads and administrator sections	Figure 17	PASS
NFR-01	Recommendation explanation tests assert score = displayed factor-point sum	Figure 12	PASS
NFR-02	test_repositories_and_database.py: FK/CHECK/UNIQUE rules, history and idempotent seed	Figure 18	PASS

CONSISTENCY CHECK Every behavioural diagram uses operations present in the class/service model. Each requirement now traces forward to a use case, implementation entry point and automated/visual verification result.

17. Implementation / Prototype and Results

A functional AISCaMS prototype was implemented using Python 3.11, Flask 3.1.3, SQLite, Jinja templates, CSS, vanilla JavaScript and pytest. The prototype contains role-specific interfaces for all five human roles and an HTTP endpoint for the IoT gateway.

Layer	Implemented artefacts
Presentation	Flask blueprints, Jinja templates, responsive CSS and browser-side JavaScript
Application	CampusFacade plus booking, maintenance, recommendation, IoT and analytics services
Domain	Users, campus resources, bookings, schedules, attendance, events, requests, readings and notifications
Patterns	Strategy, Factory, Observer, Repository and Facade used at runtime
Persistence	22-table SQLite schema with foreign keys and idempotent seed data
Verification	170 automated tests plus clean-room start-up, login, recommendation and API checks

17.1 Verification summary

Check	Method	Observed result	Status
Automated suite	pytest	170 passed	PASS
Seed run 1	python manage.py seed	370 rows	PASS
Seed run 2	repeat same command	370 rows; no duplicates	PASS
Health API	GET /api/health	HTTP 200	PASS
Student login	Demo credentials	Welcome, Athisaya	PASS
Recommendation API	POST /api/recommendations	Ranked + rejected + criteria	PASS
Booking	Submit best eligible room	BK-00007 CONFIRMED	PASS
Factory/SLA	Projector issue	IT_SUPPORT, HIGH, 6-hour SLA	PASS
IoT critical flow	AQI 268	Alert + CRITICAL ticket; duplicate suppressed	PASS
Privacy scan	Prompt/tool/credential patterns	No private instructions or tokens	PASS

VERIFIED RESULT The independent test run completed with 170/170 passing tests. Running the seed twice produced the same 370-row state, confirming idempotence.

17.2 Negative and Boundary-Case Evidence

Risk / boundary	Test evidence	Protected invariant	Result
Unauthorised role	Forbidden route/API and permission-service tests	A hidden button cannot bypass server-side permission checks.	PASS
Overlapping booking	Containment, partial overlap, exact overlap and adjacent-slot tests	Only active overlapping reservations block; adjacent slots remain valid.	PASS
Capacity/equipment	Undersized room, lab workstation and faulty/missing equipment tests	A soft ranking score never overrides feasibility or safety.	PASS
Invalid time window	Past start, end-before-start and over-eight-hour duration tests	No invalid calendar interval is persisted.	PASS
Invalid factory input	Unknown explicit category/priority and missing-text tests	Only supported concrete request types and values are created.	PASS
Illegal request state	Transition matrix tests including terminal-state attempts	History is changed only after ServiceRequest accepts the transition.	PASS
Technician ownership	Unassigned and wrong-technician update tests	A technician can advance only a request assigned to that account.	PASS
Invalid IoT source	Unknown/non-smart device, unsupported metric and non-numeric value tests	No partial reading, alert or ticket is created.	PASS
Repeated critical value	Second-reading test while matching alert remains open	Exactly one alert/ticket pair represents one unresolved incident.	PASS
Database constraints	Foreign-key, CHECK, UNIQUE and repository reconstruction tests	Persistence cannot silently violate core object relationships.	PASS

17.3 Acceptance Evaluation

Acceptance criterion	Required threshold	Observed evidence	Verdict
Role-based access	All five human roles reach the correct dashboard; protected actions reject unauthorised roles.	Role/API tests and Figures 11, 16, 17	MET
Explainable recommendation	Every candidate is hard-filtered; each eligible score exposes factors and reasons.	Recommendation tests and Figure 12	MET
Safe booking	Resource rules and conflicts are rechecked immediately before persistence.	Booking tests and Figure 13	MET
Request automation	Text produces the correct subtype, priority and subtype-specific SLA.	Factory/SLA tests and Figure 14	MET
Critical IoT handling	Critical input stores evidence and creates one assigned ticket/alert per incident.	IoT/Observer tests and Figure 15	MET
Data integrity	Schema constraints hold and a repeated seed remains exactly 370 rows.	Repository/database tests and captured run	MET
Usable evidence	Primary workflows are visible through task-focused, role-specific screens.	Figures 11-17	MET
Reproducibility	A clean environment installs, seeds, passes 170 tests and starts from documented commands.	Figure 18 and README	MET

17.4 Prototype Limitations and Production Evolution

- SQLite and single-process conflict checks are appropriate for this assessed prototype; a concurrent production service should use PostgreSQL transactions, row locking and managed migrations.
- The local Flask development server and HTTP examples demonstrate deployment structure only. Production requires a WSGI server, TLS termination, secret management, central logs, metrics, backups and recovery testing.
- Authentication is local and role based. Campus deployment should integrate SSO, enforce CSRF protection and rate limits, support stronger administrative authentication, and maintain auditable permission changes.
- The simulated HTTP IoT gateway should evolve to authenticated MQTT or a message queue with device certificates, idempotency keys, replay protection and dead-letter handling.
- Notifications are stored in-app. Email/SMS delivery should use retryable background workers rather than delaying the initiating workflow.
- The ranking policy is deliberately deterministic. Historical recalibration or a governed predictive model is justified only after collecting representative, consented data and validating fairness, drift and explainability.
- All persisted dates use one campus-local convention in the prototype. A multi-campus system should use timezone-aware UTC storage and explicit display-zone conversion.

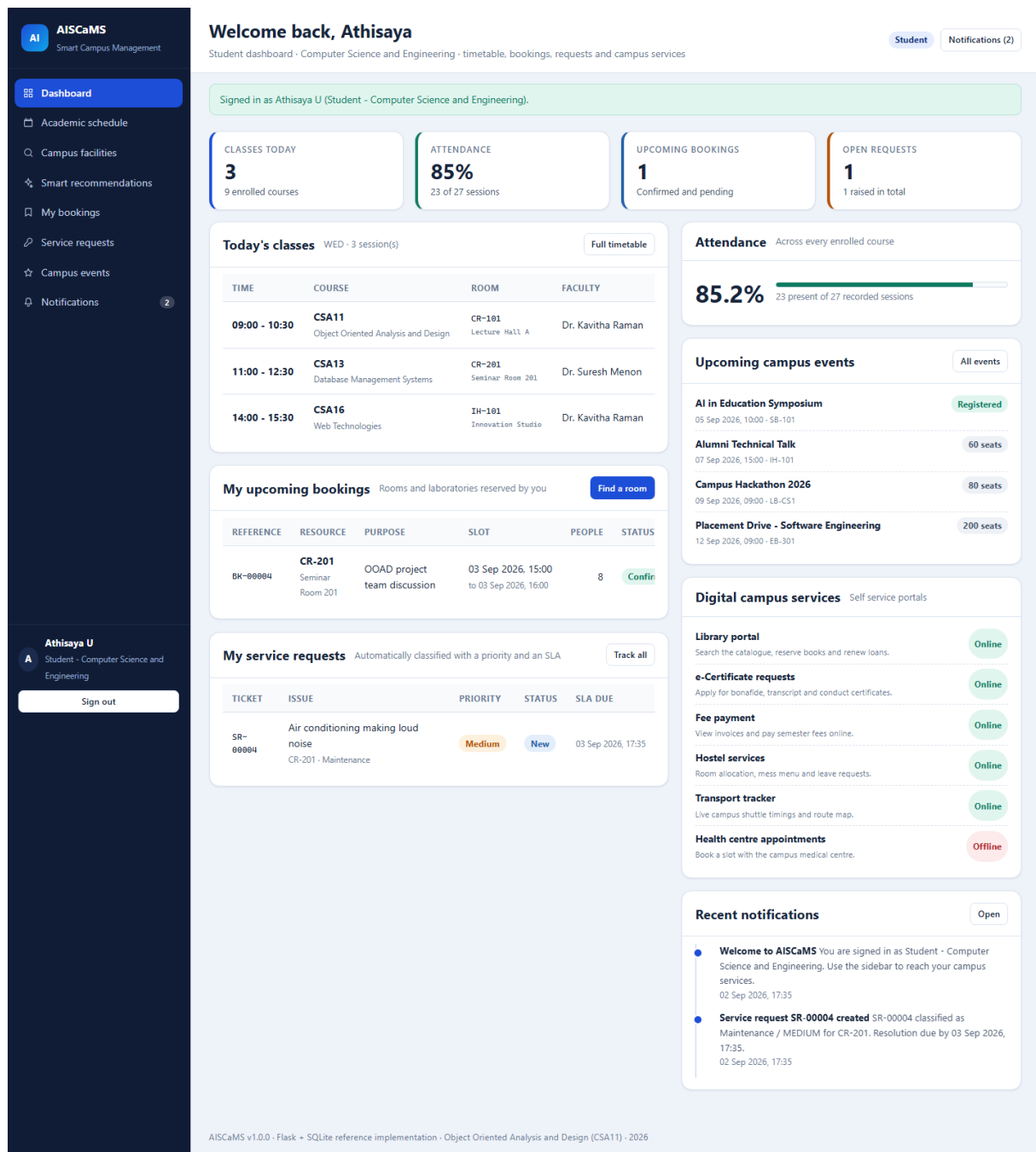


Figure 11. Student Dashboard

The student landing page combines seeded classes, 85% attendance, bookings, requests, events, digital services and notifications in one role-specific view.

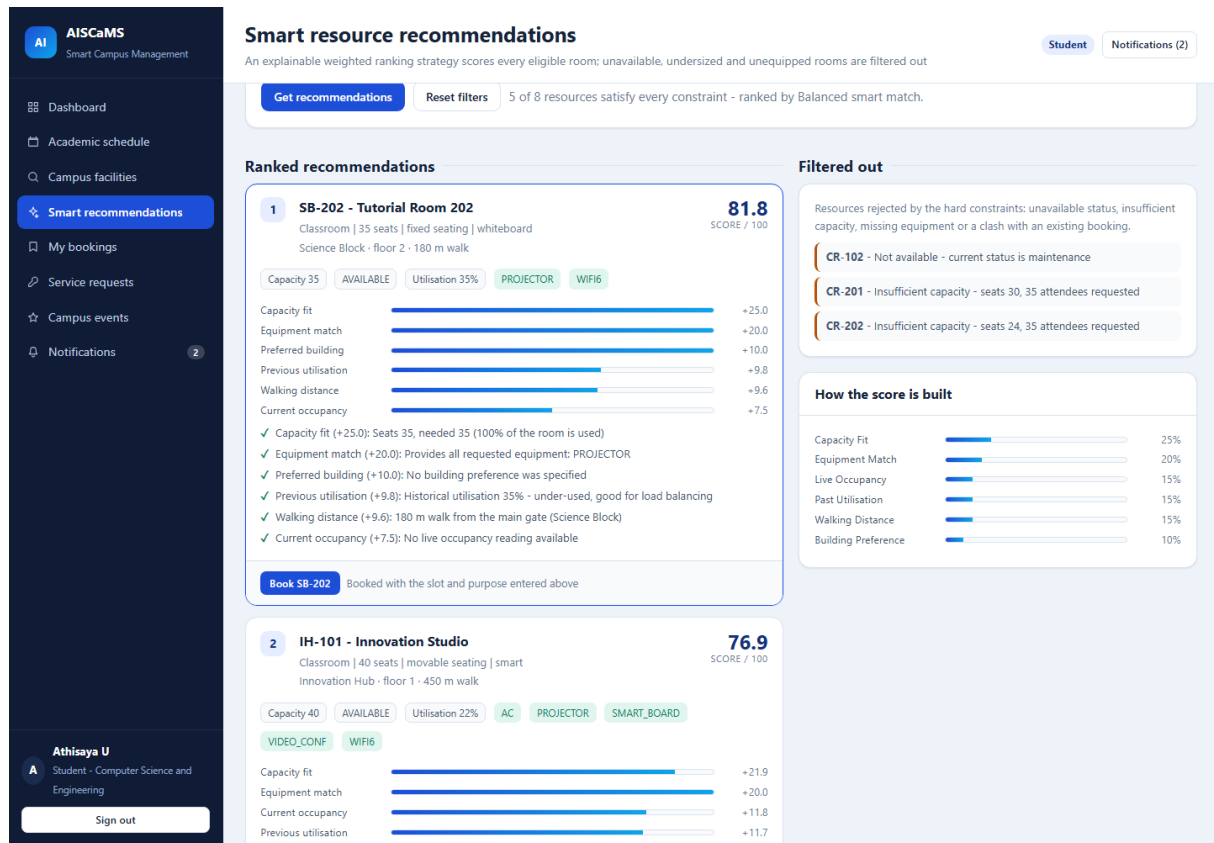


Figure 12. Explainable Smart Resource Recommendations

Eligible rooms are ranked using six weighted factors. Filtered-out rooms retain concrete rejection reasons for transparency.

AI

AISCAMS

Smart Campus Management

Dashboard

Academic schedule

Campus facilities

Smart recommendations

My bookings

Service requests

Campus events

Notifications 2

Athisaya U

Student - Computer Science and Engineering

Sign out

Smart resource recommendations

An explainable weighted ranking strategy scores every eligible room; unavailable, undersized and unequipped rooms are filtered out

Student

Notifications (2)

Booking confirmed

Booking BK-00007 confirmed.

REFERENCE

BK-00007

Status CONFIRMED

RESOURCE

SB-202

Tutorial Room 202

SLOT

02 Sep 2026, 18:00

until 02 Sep 2026, 19:00 (1 h)

ATTENDEES

24

OOAD project review meeting

A notification has been delivered to your notification centre. The slot is now blocked for other users.

Ranked recommendations

1

IH-101 - Innovation Studio

70.0

SCORE / 100

Classroom | 40 seats | movable seating | smart

Innovation Hub - floor 1 - 450 m walk

Capacity 40

AVAILABLE

Utilisation 22%

AC

PROJECTOR

SMART_BOARD

VIDEO_CONF

WIFI6

Equipment match

Capacity fit

Current occupancy

Previous utilisation

Preferred building

Walking distance

+20.0

+15.0

+11.8

+11.7

+10.0

+1.5

✓ Equipment match (+20.0): Provides all requested equipment: PROJECTOR

✓ Capacity fit (+15.0): Seats 40, needed 24 (60% of the room is used)

✓ Current occupancy (+11.8): Currently 21% occupied according to IoT sensors

✓ Previous utilisation (+11.7): Historical utilisation 22% - under-used, good for load balancing

✓ Preferred building (+10.0): No building preference was specified

✓ Walking distance (+1.5): 450 m walk from the main gate (Innovation Hub)

Book IH-101

Booked with the slot and purpose entered above

Filtered out

Resources rejected by the hard constraints: unavailable status, insufficient capacity, missing equipment or a clash with an existing booking.

CR-102 - Not available - current status is maintenance

CR-201 - Missing required equipment: PROJECTOR

CR-202 - Missing required equipment: PROJECTOR

SB-202 - Already booked during the requested slot (18:00-19:00)

How the score is built

Capacity Fit

Equipment Match

Live Occupancy

Past Utilisation

Walking Distance

Building Preference

25%

20%

15%

15%

15%

10%

Figure 13. Successful Resource Booking

Booking BK-00007 is confirmed for SB-202. The recommendation list is recalculated so the new time conflict is immediately visible.

AI

AISCAMS

Smart Campus Management

Dashboard

Academic schedule

Campus facilities

Smart recommendations

My bookings

Service requests

Campus events

Notifications 3

Athisaya U

Student - Computer Science and Engineering

Sign out

Service requests

Report a campus issue and track it; the factory classifies the category, the priority and the SLA automatically

Student

Notifications (3)

What is wrong?

The ceiling projector in the lecture hall does not switch on and the class cannot continue. Urgent support is required before the next session.

Category

Detect automatically

Priority

Detect automatically

Submit request

Keyword rules choose the category; escalation words raise the priority.

Request SR-00008 created

Request SR-00008 created and classified as IT support / HIGH (SLA 6 hours).

CATEGORY

IT SUPPORT

Campus IT support

PRIORITY

HIGH

Assigned automatically

SLA

6 hours

Due 02 Sep 2026, 23:36

STATUS

NEW

CR-101 - ON TRACK

The administrator has been notified and can assign the ticket to a maintenance technician.

Resolution targets (SLA)

Safety

1 / 4 / 12 / 24 h

IT support

2 / 6 / 16 / 48 h

Maintenance

2 / 8 / 24 / 72 h

Housekeeping

2 / 8 / 12 / 36 h

Read as critical / high / medium / low priority.

My requests 2 ticket(s) raised by you

TICKET	ISSUE	CATEGORY	PRIORITY	STATUS	SLA
SR-00008	Projector not working in Lecture Hall A CR-101 - raised 02 Sep 2026, 17:36	IT support	high	new	on track 6h - 02 Sep 2026, 23:36
SR-00004	Air conditioning making loud noise CR-201 - raised 02 Sep 2026, 17:35	Maintenance	Medium	New	On Track 24h - 03 Sep 2026, 17:35

Figure 14. Automatically Classified Service Request

The request is classified as IT support with HIGH priority, a six-hour SLA and NEW status using the Factory pattern.

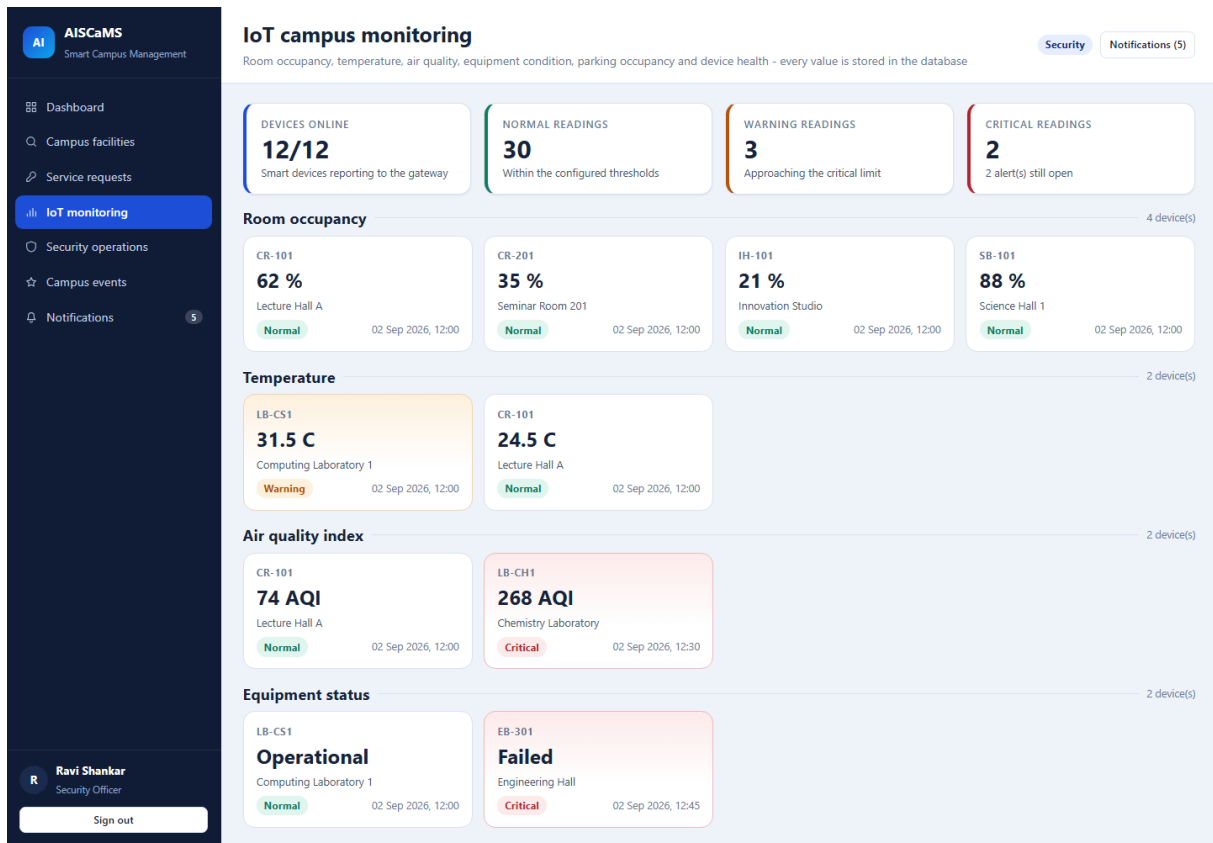


Figure 15. IoT Monitoring and Critical AQI Alert

The screen reports 12/12 devices online, live metrics and an AQI reading of 268 classified as critical.

AI

AISCaMS

Smart Campus Management

Dashboard

Campus facilities

Service requests

Maintenance workspace

IoT monitoring

Notifications 6

Mohan Das

Maintenance Technician - Electrical and IT facilities

Sign out

Maintenance workflow

Work the assigned queue: start the job, resolve it and close it. Only transitions allowed by the state machine are offered

Maintenance

Notifications (6)

OPEN TICKETS

2

2 ticket(s) in the queue

IN PROGRESS

2

Work started on site

SLA BREACHED

1

Past the promised resolution time

RESOLVED OR CLOSED

0

Completed work

Service request queue

Priority order: critical tickets first, then by age

TICKET	ISSUE	RESOURCE	PRIORITY	STATUS	SLA	WORKFLOW
SR-00006 IOT	LB-CH1: air quality index critical reading Automatically generated from IoT device IOT-AIR-CH1. Air quality index measured 268 AQI which crossed the crit...	LB-CH1 Safety	Critical	In Progress	Breached 1h · due 02 Sep 2026, 13:30	Mark resolved
	▼ History (3)					
	New Raised automatically by IoT gateway 2026-09-02 12:30:00 Assigned Auto-assigned to Mohan Das by the IoT gateway 2026-09-02 17:35:36 In Progress Updated from the maintenance workspace 2026-09-02 17:36:21 · Mohan Das					
SR-00001 USER	Projector not working in Lecture Hall A The ceiling projector in CR-101 is not working since this morning and the lecture could not be delivered with ...	CR-101 IT support	High	In Progress	On Track 6h · due 02 Sep 2026, 23:35	Mark resolved
	▼ History (3)					
	New Request created 2026-09-02 17:35:35 · Dr. Kavitha Raman Assigned Assigned to Mohan Das 2026-09-02 17:35:35 · Vikram Rao In Progress Technician on site, replacing the projector lamp. 2026-09-02 17:35:35 · Mohan Das					

Update equipment condition

Equipment marked faulty or out of service is excluded from recommendations

Figure 16. Maintenance Request Workflow

The assigned technician views an in-progress request, complete history, SLA information and only the next legal transition.

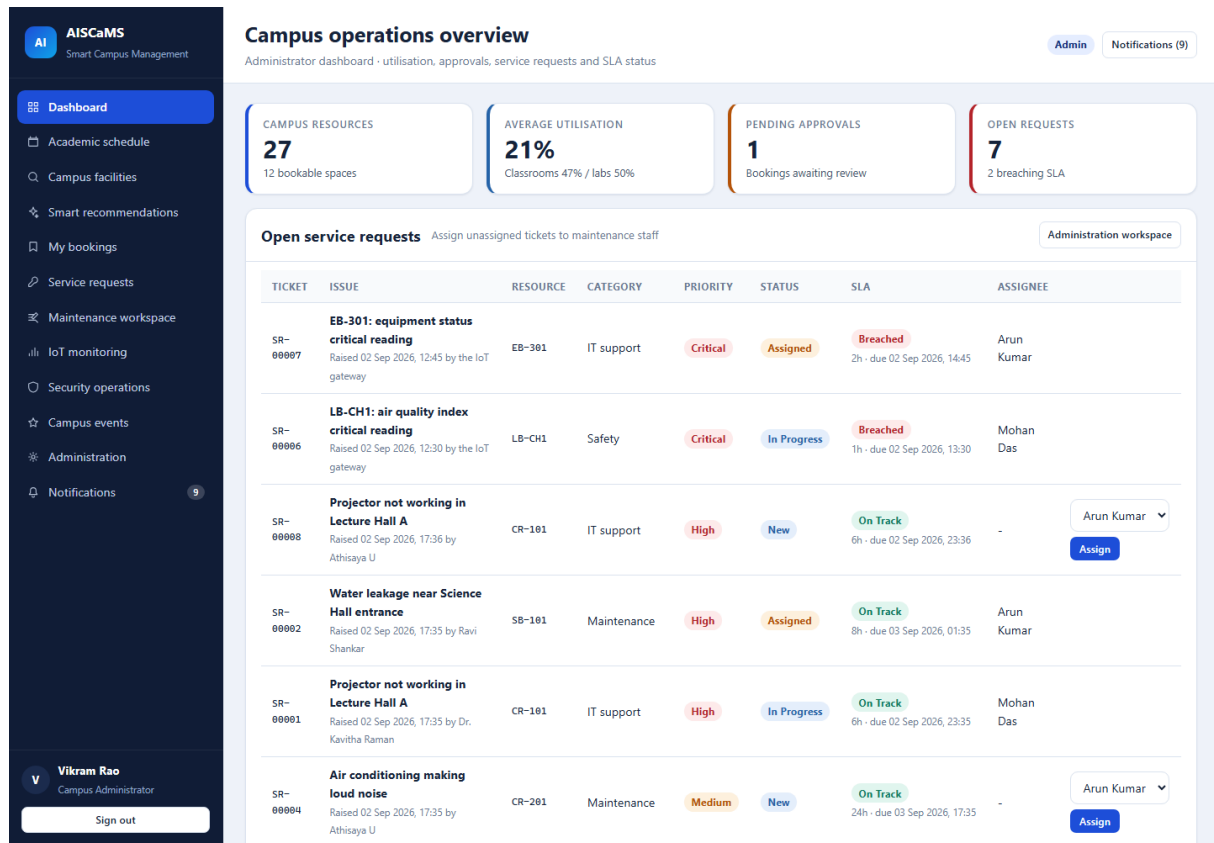
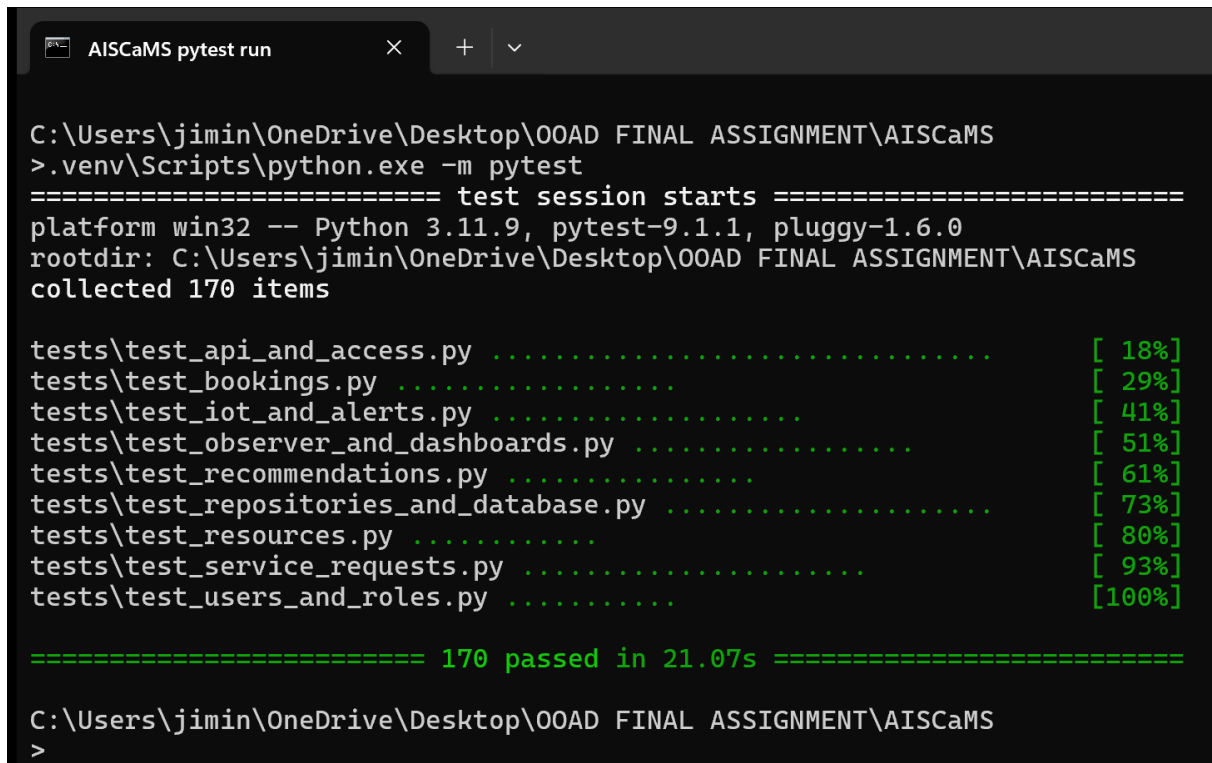


Figure 17. Administrator Dashboard

Administrative analytics summarise 27 resources, utilisation, pending approvals, seven open requests and two SLA breaches.

A terminal window titled "AISCAMS pytest run" showing the output of a pytest command. The terminal displays the directory path, the command used, the pytest version and platform information, the number of items collected, and a list of test files with their progress percentages. The final summary shows that all 170 tests passed in 21.07 seconds.

```
AISCAMS pytest run
C:\Users\jimin\OneDrive\Desktop\OOAD FINAL ASSIGNMENT\AISCAMS
>.venv\Scripts\python.exe -m pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\jimin\OneDrive\Desktop\OOAD FINAL ASSIGNMENT\AISCAMS
collected 170 items

tests\test_api_and_access.py ..... [ 18%]
tests\test_bookings.py ..... [ 29%]
tests\test_iot_and_alerts.py ..... [ 41%]
tests\test_observer_and_dashboards.py ..... [ 51%]
tests\test_recommendations.py ..... [ 61%]
tests\test_repositories_and_database.py ..... [ 73%]
tests\test_resources.py ..... [ 80%]
tests\test_service_requests.py ..... [ 93%]
tests\test_users_and_roles.py ..... [100%]

===== 170 passed in 21.07s =====
C:\Users\jimin\OneDrive\Desktop\OOAD FINAL ASSIGNMENT\AISCAMS
>
```

Figure 18. Automated Test Results

The authentic terminal capture records the complete pytest run with all 170 tests passing.

18. Reflection on Design Decisions and Learning Outcomes

The most important design decision was to keep recommendations explainable. A trained model would have added complexity without dependable campus data, whereas the deterministic strategy makes every point and rejection auditable. Separating hard filters from soft ranking also prevents a high score from overriding safety, capacity or conflict rules.

The request subsystem demonstrated why behaviour should remain inside objects instead of being scattered across interface code. ServiceRequest protects legal transitions, each subtype owns its SLA matrix, and the Factory converts free text or IoT readings into a valid concrete object. The same workflow can therefore serve students, administrators, technicians and devices without duplicating creation logic.

The prototype also strengthened my understanding of consistency across UML views. Use cases identify goals, sequence diagrams allocate messages, the class diagram supplies the receiving operations, activity/state diagrams constrain behaviour, and package/component/deployment views explain where those responsibilities execute. Automated tests then provide evidence that the implementation follows those models.

18.1 Critical Design Trade-offs

Decision	Reasoned choice	Consequence / boundary
Deterministic strategy vs machine learning	The available data are seeded and too small to justify training. A declared weighted strategy is reproducible, testable and explainable.	The term AI-enabled is limited to transparent decision automation; future learning requires governed real data and validation.
Layered model vs direct route-to-SQL	Domain objects protect invariants, services coordinate use cases and repositories isolate persistence.	The extra classes add initial structure, but substantially reduce duplicated rules and improve test isolation.
Observer events vs direct side effects	Notification, alert and audit reactions can change independently of the initiating booking/request/IoT workflow.	In-process observers are simple but not durable across process failure; production should add a transactional event/outbox mechanism.
SQLite vs server database	SQLite gives a reproducible, zero-service academic package and supports all demonstrated constraints.	It is not the final choice for concurrent multi-host traffic; transactional PostgreSQL is the planned evolution.

18.2 Defects Found and Lessons Applied

Verification exposed two defects that the design narrative alone would not have revealed. First, SQLite `datetime('now')` values were UTC while Python-created timestamps used campus local time, so request history appeared several hours out of order. The schema was corrected to use one local-time convention for the prototype, and the future-production requirement now specifies timezone-aware UTC storage and display conversion.

Second, score-factor bars appeared empty even though the calculated widths were correct. The bars were inline span elements, so CSS ignored their width. Making the element render as an inline block restored the visual explanation. This reinforced that explainability is not achieved merely by calculating reasons: the evidence must also be rendered accurately and covered by end-to-end inspection.

18.3 Limitations and Future Learning

- The prototype demonstrates OOAD consistency and deterministic automation, not production-scale concurrency or a trained predictive model.
- A larger evaluation should measure task completion, accessibility, response time and recommendation usefulness with representative campus users.
- IoT reliability work should explore authenticated messaging, intermittent connectivity, event replay and failure recovery rather than assuming uninterrupted HTTP delivery.
- Future design iterations should use migrations, timezone-aware persistence and an outbox/event queue while preserving the current domain and pattern contracts.

18.4 SDG Contribution

Mapped SDG	Design mechanism	Observable contribution
SDG 4 - Quality Education	Resource discovery, schedules, attendance	Students can locate suitable learning spaces and

Mapped SDG	Design mechanism	Observable contribution
	and digital-service access	see current academic information in one dashboard.
SDG 9 - Industry, Innovation and Infrastructure	Explainable decision automation, modular services and IoT condition monitoring	Critical readings become traceable alerts/tickets, while patterns support maintainable infrastructure software.
SDG 11 - Sustainable Cities and Communities	Utilisation-aware booking, maintenance workflow and live parking visibility	Shared facilities are used more effectively and infrastructure incidents receive accountable responses.

18.5 Learning Outcomes Achieved

- CO1 - Analysed actors, objects, responsibilities, relationships and constraints in a multi-domain problem.
- CO2 - Applied structural, behavioural and architectural UML diagrams consistently with the implemented object model.
- CO2 - Used encapsulation, inheritance, polymorphism, cohesion and coupling as reasoned design tools rather than labels.
- CO3 - Justified Strategy, Factory, Observer, Repository and Facade through runtime variation, creation and collaboration needs.
- CO3 - Connected requirements to use cases, classes, entry points, implementation evidence and executable verification.

19. GitHub Upload

The complete, privacy-reviewed submission is available in the public repository below. It contains the reproducible prototype, automated verification evidence, final report and the ten supplied Umbrello diagram exports. Generated databases, caches, virtual environments and internal working files are excluded.

Repository: github.com/athisayaprema2007-hue/CSA11-AISCaMS-Smart-Campus

Repository area	Contents
app/	Domain classes, patterns, repositories, services, routes, templates, schema and seed data
tests/	170 automated tests covering users, resources, bookings, recommendations, requests, IoT, observers and APIs
docs/screenshots/	Eight authentic prototype and terminal screenshots
docs/uml/	Ten black-and-white UML evidence images exported from Umbrello
docs/report/	Final PDF submission and editable Word report
docs/	Implementation handoff, report package and captured test results
Root files	README, requirements, run scripts, management commands and .gitignore

REPRODUCTION Create a Python virtual environment, install requirements.txt, run python manage.py seed, execute pytest -q, and start the application with python run.py.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	10	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	15	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.	Classes, attributes, methods, or relationships are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.
UML Architectural Modelling & Design	15	Package, Component, and Deployment Diagrams clearly represent modular	Architectural diagrams are mostly correct with minor	Basic architectural representation is provided but	Architectural diagrams are incomplete, inappropriate, or

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Evaluation (CO3)		structure, system components, dependencies, and deployment environment with strong justification.	omissions or lacks clarity or modelling errors.	sufficient justification.	inconsistent with the system design.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability & Overall Design Quality (CO3)	10	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated, but important links are missing.	Major inconsistencies exist between requirements, use cases, classes, and UML diagrams.
Reflection, SDG Relevance & Learning Outcomes	10	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and SDG relevance.	Reflection is missing, superficial, or unrelated to the assignment and SDGs.
Total	100				